



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

پروژه درس معماری کامپیوتر

تحلیل نقصان‌های برخورد و طراحی معماری حافظه نهان قربانی

نگارش

نفیسه زارعی

آرش اکبری

آیناز رحمانی

استاد

دکتر حمید سربازی آزاد

مرداد ۱۴۰۵

چکیده

در این پروژه به بررسی و حل مشکل افت عملکرد ناشی از نقصان‌های برخورد در حافظه‌های نهان با روش نگاشت مستقیم پرداخته شده است. تداخل آدرس‌ها در این ساختار منجر به پدیده تلاطم و مراجعه‌های مکرر و پرهزینه به حافظه اصلی می‌شود. برای رفع این چالش، یک حافظه نهان قربانی تمام‌انجمی طراحی و بین سطح اول حافظه نهان و سطح پایین‌تر قرار داده شد. این معماری در شبیه‌ساز آکیتا پیاده‌سازی شده و عملکرد آن با استفاده از یک برنامه محک با دسترسی‌های متناوب مورد ارزیابی قرار گرفت. نتایج شبیه‌سازی و بررسی شمارنده‌های سخت‌افزاری نشان می‌دهد که افزودن این پیمان به مدیریت صحیح جابه‌جایی دوطرفه، ترافیک حافظه اصلی را به شکل چشمگیری کاهش داده و زمان دسترسی میانگین به حافظه را بهبود می‌بخشد.

کلیدواژه‌ها: حافظه نهان قربانی، نقصان برخورد، نگاشت مستقیم، تلاطم، زمان دسترسی میانگین.

فهرست مطالب

۱	مقدمه	۱
۳	مفاهیم پایه و پیش زمینه	۲
۳	۱-۲ سلسله مراتب حافظه	۳
۳	۲-۲ انواع نقصان در حافظه نهان	۳
۴	۱-۲-۲ نقصان اجباری	۴
۴	۲-۲-۲ نقصان ظرفیتی	۴
۴	۳-۲-۲ نقصان برخورد	۴
۵	۳ ساختار پایه و شبیه سازی وضعیت تلاطم	۳
۵	۱-۳ معماری پایه در شبیه ساز آکیتا	۵
۵	۲-۳ برنامه محک و ایجاد تلاطم	۵
۶	۴ طراحی و معماری حافظه نهان قربانی	۴
۶	۱-۴ ساختار تمام انجمی و سیاست جایگزینی	۶
۷	۲-۴ مکانیزم تبادل دوطرفه و مسیریابی پیام ها	۷
۸	۵ ارزیابی و تحلیل نتایج شبیه سازی	۵
۸	۱-۵ مقایسه ترافیک حافظه و تاخیر سامانه	۸
۸	۲-۵ تحلیل ریاضی زمان دسترسی میانگین	۸

۶ نتیجه گیری

۹

مراجع

۱۰

فصل ۱

مقدمه

شکاف فزاینده بین سرعت پردازنده‌ها و حافظه اصلی یکی از چالش‌های اساسی در معماری کامپیوتر است. برای پر کردن این شکاف از سلسله‌مراتب حافظه نهان^۱ استفاده می‌شود. با این حال محدودیت‌هایی که در ساختارهای نگاشت حافظه وجود دارد، چالش‌های جدیدی را به وجود می‌آورد. به عنوان مثال، به دلیل سادگی و سرعت بالا در پیاده‌سازی، استفاده از حافظه‌های نهان با نگاشت مستقیم^۲ یا انجمنی با درجه پایین بسیار رایج است اما این روش باعث بروز نقصان‌های برخورد^۳ می‌شود. در این حالت ممکن است چندین آدرس پرکاربرد دقیقاً به یک خط از حافظه نهان اختصاص یابند. حتی اگر ظرفیت کلی حافظه برای نگهداری داده‌ها کافی باشد، این تداخل باعث می‌شود داده‌ها پیوسته از حافظه اخراج^۴ شوند.

این رفتار ناکارآمد که به عنوان پدیده تلاطم^۵ شناخته می‌شود، سامانه را مجبور می‌کند تا داده‌ها را بارها از سطوح پایین‌تر واکشی کند. این فرایند نه تنها ترافیک بیهوده‌ای در شبکه ارتباطی حافظه ایجاد می‌کند، بلکه تاخیر سنگینی را به پردازنده تحمیل می‌نماید. برای حل این مشکل، ایده اضافه کردن یک بافر بسیار کوچک و تمام‌انجمنی^۶ به عنوان حافظه نهان قربانی^۷ مطرح می‌شود که داده‌های اخراجی از سطح اول را پیش از انتقال به سطح پایین‌تر در خود نگه می‌دارد.

در این پروژه روند ناکارآمد ذکر شده ابتدا در قالب یک معماری پایه شبیه‌سازی و مدل‌سازی شده است تا میزان هدررفت عملکرد سامانه اندازه‌گیری شود. سپس با طراحی یک پیمانۀ جدید و تغییر منطق مسیریابی

Cache^۱

Direct-Mapped^۲

Conflict Misses^۳

Evict^۴

Thrashing^۵

Fully Associative^۶

Victim Cache^۷

پیام‌ها^۸، سامانه بهینه شده است. هدف نهایی این گزارش تحلیل تاثیر این معماری جدید بر موازنه میان هزینه ساخت‌افزایی و زمان دسترسی میانگین^۹ برنامه‌ها و همچنین بررسی روند جلوگیری از بن‌بست^{۱۰} در تبادل هم‌زمان پیام‌ها است.

فصل ۲

مفاهیم پایه و پیش زمینه

در این فصل مفاهیم پایه‌ای مرتبط با ساختار حافظه نهان^۱ و چالش‌های عملکردی آن بررسی می‌شود. این مفاهیم برای درک بهتر اهداف این پروژه از جمله تشخیص انواع نقصان‌ها و طراحی معماری حافظه نهان قربانی^۲ ضروری هستند.

۱-۲ سلسله مراتب حافظه

در معماری‌های پایه و مرسوم، زمانی که پردازنده به داده‌ای نیاز دارد ابتدا آن را در حافظه نهان سطح اول جستجو می‌کند. اگر داده به دلیل نقصان در این سطح یافت نشود، از سطوح پایین‌تر واکشی می‌شود. هنگام ورود یک بلاک جدید به حافظه نهان سطح اول، ممکن است بلاک قدیمی موجود در آن جایگاه بلافاصله اخراج^۳ شده و به سطوح پایین‌تر فرستاده شود. در صورتی که پردازنده دوباره به همان بلاک اخراج شده نیاز پیدا کند، سامانه باید تاخیر سنگین خواندن دوباره آن از سطوح پایین‌تر را تحمل کند.

۲-۲ انواع نقصان در حافظه نهان

هنگام دسترسی پردازنده به حافظه، در صورتی که داده مورد نظر در سطح فعلی موجود نباشد، پدیده‌ای به نام نقصان اتفاق می‌افتد. نقصان‌ها به طور کلی به سه دسته اصلی تقسیم می‌شوند که درک تفاوت میان آن‌ها برای تحلیل عملکرد سیستم اهمیت بالایی دارد [cite: ۱].

^۱ Cache Memory

^۲ Victim Cache

^۳ Evict

۱-۲-۲ نقصان اجباری

^۴ این نوع نقصان زمانی رخ می‌دهد که پردازنده برای اولین بار به یک آدرس از حافظه دسترسی پیدا می‌کند. از آنجا که داده مربوط به این آدرس پیش از این خوانده نشده است، وقوع این نقصان و واکنشی آن از سطوح پایین‌تر اجتناب‌ناپذیر است.

۲-۲-۲ نقصان ظرفیتی

^۵ نقصان ظرفیتی زمانی اتفاق می‌افتد که گنجایش کلی حافظه نهان برای نگهداری تمام داده‌های مورد نیاز برنامه کافی نباشد [cite: ۱]. در پیاده‌سازی این پروژه و در بخش ایجاد کننده درخواست‌ها، این پدیده با واکنشی آدرس‌هایی خارج از گستره ظرفیت حافظه نهان به صورت عملی مدل‌سازی شده است [cite: ۴].

۳-۲-۲ نقصان برخورد

^۶ این مشکل به طور ویژه در حافظه‌های نهانی رخ می‌دهد که با روش نگاشت مستقیم^۷ یا انجمنی^۸ با درجه پایین کار می‌کنند [cite: ۱]. به دلیل محدودیت‌هایی که در ساختارهای نگاشت حافظه وجود دارد، ممکن است چندین آدرس پرکاربرد به یک خط مشخص از حافظه نهان اختصاص یابند [cite: ۱]. حتی اگر ظرفیت کلی حافظه نهان برای نگهداری داده‌ها کافی باشد، تداخل آدرس‌ها باعث می‌شود داده‌ها پیوسته از حافظه اخراج شده و پدیده‌ای به نام تلاطم^۹ رخ دهد [cite: ۱]. این اتفاق سامانه را مجبور می‌کند تا داده‌ها را بارها از سطوح پایین‌تر حافظه واکنشی کند که جریمه زمانی و تاخیر قابل توجهی را به همراه دارد [cite: ۱]. در کدهای شبیه‌سازی این پروژه، این رفتار ناکارآمد با ایجاد حلقه‌ای از درخواست‌های مکرر بر روی آدرس‌های متعارض پیاده‌سازی شده است [cite: ۴].

Compulsory Miss^۴

Capacity Miss^۵

Conflict Miss^۶

Direct-Mapped^۷

Associative^۸

Thrashing^۹

فصل ۳

ساختار پایه و شبیه‌سازی وضعیت تلاطم

میو

۱-۳ معماری پایه در شبیه‌سازی آکیتا

میو

۲-۳ برنامه محک و ایجاد تلاطم

میو

فصل ۴

طراحی و معماری حافظه نهان قربانی

در این فصل به بررسی ساختار و پیاده‌سازی حافظه نهان قربانی می‌پردازیم که راه حلی برای کاهش ترافیک اضافی حافظه است [cite: ۱]. این پیمان^۱ به عنوان یک بافر بسیار کوچک بین حافظه نهان سطح اول و سطوح پایین‌تر قرار می‌گیرد تا از ارسال درخواست‌های پرهزینه جلوگیری کند [cite: ۱].

۴-۱ ساختار تمام‌انجمنی و سیاست جایگزینی

حافظه نهان قربانی پیاده‌سازی شده در این پروژه دارای یک ساختار تمام‌انجمنی^۲ است [cite: ۱]. بر اساس مشخصات تعریف شده در کدهای شبیه‌ساز، این حافظه به طور پیش‌فرض دارای گنجایش ۸ بلاک^۳ است [cite: ۵]. اندازه هر بلاک برابر با ۶۴ بیت در نظر گرفته شده است تا با اندازه بلاک‌های حافظه نهان سطح اول برابری کند و از بروز خطای عدم تطابق در آدرس‌دهی جلوگیری شود [cite: ۵]. هر جایگاه در این بافر با استفاده از یک ساختار داده مشخص مدل‌سازی شده است که شامل برچسب^۴، بیت اعتبار^۵، داده‌ها و متغیری برای ثبت زمان آخرین دسترسی است [cite: ۵].

برای مدیریت سرریز گنجایش محدود این حافظه، از سیاست جایگزینی کمترین استفاده اخیر^۶ بهره گرفته شده است [cite: ۱، ۵]. هنگام ورود یک بلاک اخراجی جدید از سطح بالا، سیستم ابتدا بررسی می‌کند که آیا آدرس مورد نظر از پیش در بافر وجود دارد یا خیر [cite: ۵]. در صورت نبود آن، به دنبال

^۱Module

^۲Fully Associative

^۳Block

^۴Tag

^۵Valid Bit

^۶Least Recently Used (LRU)

یک جایگاه خالی می‌گردد و اگر تمام جایگاه‌ها پر باشند، با مقایسهٔ زمان آخرین دسترسی تمام بلاک‌ها، قدیمی‌ترین آن‌ها را انتخاب کرده و بلاک جدید را جایگزین آن می‌کند [cite: ۵].

۴-۲ مکانیزم تبادل دوطرفه و مسیریابی پیام‌ها

در معماری جدید، منطق مسیریابی پیام‌ها^۷ به گونه‌ای تغییر یافته است که بلاک‌های اخراجی از حافظهٔ نهان سطح اول به جای انتقال به حافظهٔ اصلی، به صورت مستقیم وارد درگاه بالایی حافظهٔ قربانی می‌شوند [cite: ۱، ۵]. این پیام‌های اخراجی در قالب درخواست‌های نوشتن دریافت می‌شوند و سامانه به محض دریافت آن‌ها، یک پاسخ اتمام نوشتن برای سطح بالاتر ارسال می‌کند تا بافر آن سطح بدون معطلی آزاد شود [cite: ۵].

یکی از فرایندهای کلیدی در این معماری، مدیریت جابه‌جایی دوطرفه^۸ است [cite: ۱]. هنگامی که پردازنده به داده‌ای نیاز دارد که در حافظهٔ سطح اول نیست، یک درخواست خواندن به حافظهٔ قربانی ارسال می‌شود [cite: ۵]. اگر آدرس درخواستی در این بافر یافت شود (وقوع اصابت)، دادهٔ مورد نیاز به سرعت به سطح اول برمی‌گردد [cite: ۱، ۵]. این حافظه از سیاست انحصاری^۹ پیروی می‌کند و این بدان معنا است که به محض بازگرداندن داده به سطح بالاتر، نسخهٔ موجود در حافظهٔ قربانی نامعتبر می‌شود تا مالکیت داده به صورت کامل به سطح اول منتقل گردد [cite: ۵].

در صورت عدم اصابت، درخواست به صورت ناهمگام^{۱۰} به درگاه پایینی (حافظهٔ اصلی) فرستاده می‌شود و در یک صف انتظار قرار می‌گیرد [cite: ۵]. تمام این فرایندها و تبادل پیام‌ها به کمک ماشین حالت^{۱۱} و با بررسی مداوم شرایط ارسال پیام مدیریت می‌شوند تا از بروز چالش بن‌بست^{۱۲} در شبیه‌ساز جلوگیری شود [cite: ۱، ۵]. افزون بر این، برای مدل‌سازی تاخیر سخت‌افزاری، یک زمان تاخیر دو سیکلی برای پردازش تراکنش‌های ورودی در نظر گرفته شده است که با استفاده از یک شمارنده کاهش می‌یابد تا به صفر برسد و سپس پردازش ادامه یابد [cite: ۵].

Message Routing^۷

Swap^۸

Exclusive Policy^۹

Asynchronous^{۱۰}

State Machine^{۱۱}

Deadlock^{۱۲}

فصل ۵

ارزیابی و تحلیل نتایج شبیه‌سازی

میو

۵-۱ مقایسه ترافیک حافظه و تاخیر سامانه

میو

۵-۲ تحلیل ریاضی زمان دسترسی میانگین

میو

فصل ۶

نتیجه گیری

میور

References